

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 3.4

Create a Draft Order and Fetch Its Invoice URL

Module	3 — Shopify Platform, Storefront API, Admin API & Native AI
Topic	3.4 — Admin API (GraphQL) — Orders, Draft Orders & Customers
Estimated time	30–40 minutes
Tools needed	Terminal, Node.js 20+, a code editor, a web browser
Prerequisite	Lab 3.2 complete (a working <code>adminClient.js</code> + <code>.env</code>); ideally Lab 3.3 complete too, for a real variant ID to reuse

What You're Building

GOAL

A real draft order created against your dev store via the Admin API, with its `invoiceUrl` captured two ways — once directly from the create mutation, and once by fetching it back afterward — then opened in a browser to confirm it's a genuine, payable Shopify invoice.

This reuses `adminClient.js` from Lab 3.2 exactly as it was — no new scopes needed, since `write_draft_orders` was already declared there. If that project folder still exists, work inside it directly.

Steps 1–2: Get Set Up and Grab a Variant

1 Reuse your Lab 3.2 project

Open the same project folder from Lab 3.2 (with `adminClient.js` and a working `.env`). Confirm it still authenticates:

TERMINAL

```
node test-admin.js
```

What you'll see: the same `shop { name email }` JSON from Lab 3.2. If it fails, the token helper's cached token has simply expired — it re-requests one automatically, so just run it again.

2 Get a real product variant ID

If you still have a variant ID from Lab 3.3, reuse it — otherwise fetch one straight from the Admin API:

GET-VARIANT.JS

```
const { adminRequest } = require("../adminClient");

adminRequest(`
  query {
    products(first: 1) {
      edges { node { title variants(first: 1) { edges { node { id } } } } }
    }
  }
`).then((data) => console.log(JSON.stringify(data, null, 2))).catch(console.error);
```

TERMINAL

```
node get-variant.js
```

What you'll see: a product title and a variant `id` shaped like `gid://shopify/ProductVariant/...`. Copy that value — it's used in Step 3.

Steps 3–4: Create the Draft Order, Then Fetch It Back

3 Create the draft order

CREATE-DRAFT-ORDER.JS

```
const { adminRequest } = require("../adminClient");

async function main() {
  const variantId = "gid://shopify/ProductVariant/PASTE_YOUR_VARIANT_ID";

  const data = await adminRequest(`
    mutation createDraftOrder($variantId: ID!) {
      draftOrderCreate(input: { lineItems: [{ variantId: $variantId, quantity: 1 }] }) {
        draftOrder { id name invoiceUrl }
        userErrors { field message }
      }
    }
  `, { variantId });

  if (data.draftOrderCreate.userErrors.length) {
    throw new Error(JSON.stringify(data.draftOrderCreate.userErrors));
  }

  console.log("Draft order created:", data.draftOrderCreate.draftOrder);
}

main().catch(console.error);
```

TERMINAL

```
node create-draft-order.js
```

What you'll see: a printed object with a real `id` (shaped like `gid://shopify/DraftOrder/...`), a `name` like `#D1`, and an `invoiceUrl`. Copy the `id` — it's used in Step 4.

4 Fetch the invoice URL back separately

Step 3 already returned an `invoiceUrl` inline, but a real agent often needs to look up an *existing* draft order later, not just at the moment it's created. Prove that works too:

GET-DRAFT-ORDER.JS

```
const { adminRequest } = require("../adminClient");

async function main() {
```

```
const draftOrderId = "gid://shopify/DraftOrder/PASTE_ID_FROM_STEP_3";

const data = await adminRequest(`
  query getDraftOrder($id: ID!) {
    draftOrder(id: $id) {
      id
      name
      invoiceUrl
      totalPrice
      lineItems(first: 5) { edges { node { title quantity } } }
    }
  }
`, { id: draftOrderId });

console.log("Fetched back:", JSON.stringify(data.draftOrder, null, 2));
}

main().catch(console.error);
```

TERMINAL

```
node get-draft-order.js
```

What you'll see: the same `invoiceUrl` as Step 3, plus the line item and price you'd expect — confirming the draft order persisted and can be re-queried independently of the create call.

Node quirk worth knowing: `console.log("label:", someObject)` only prints nested objects 2 levels deep by default — anything deeper collapses to `[Object]`, which is exactly why this script wraps the result in `JSON.stringify(..., null, 2)` instead. Worth remembering any time your own script's output looks suspiciously empty.

Steps 5–6: Verify in the Browser, Then Query Customers

5 Open the invoice URL and confirm it's real

The invoice URL lives on your store's own `.myshopify.com` domain, so it's subject to the same storefront password protection as Lab 3.3's checkout link. **Unlock the store first:** visit `https://your-store.myshopify.com/` in your browser and enter the storefront password, *then* open the `invoiceUrl` from Step 3 or 4 in a new tab.

Why the order matters: every Shopify dev store is password-protected with no way to turn it off. A fresh invoice or checkout link opened in a browser that hasn't unlocked the store yet hits the password gate first — and that redirect is known to drop the original destination, landing you on the plain homepage instead. Unlocking the store first avoids the gate firing at all.

What you'll see: a real, payable Shopify invoice page listing the product and quantity from Step 3, with a "Pay now" button — the exact page a customer would receive to pay for a merchant- or agent-initiated sale.

6 Query real customer data

Run one more query to satisfy the module's requirement for real customer or inventory data:

GET-CUSTOMERS.JS

```
const { adminRequest } = require("../adminClient");

adminRequest(`
  query {
    customers(first: 3) {
      edges { node { id firstName lastName email numberOfOrders } }
    }
  }
`).then((data) => console.log(JSON.stringify(data, null, 2))).catch(console.error);
```

TERMINAL

```
node get-customers.js
```

What you'll see: real customer records from your dev store's seeded test data (or an empty array if none exist yet — still a valid, successful response). If this instead fails with an approval-style error, revisit Lab 3.2's Protected Customer Data step.

Acceptance checklist

- ☐ `draftOrderCreate` ran successfully against your dev store
- ☐ The response includes a usable `invoiceUrl`
- ☐ The same `invoiceUrl` was also fetched back with a separate `draftOrder(id: ...)` query, not just the create response
- ☐ At least one customer or inventory query returns real dev-store data
- ☐ Can explain when to use `draftOrderCreate` vs. `orderCreate`

Troubleshooting

Symptom	Likely cause	Try this
<code>draftOrderCreate</code> returns a <code>userErrors</code> entry about the variant	The variant ID is stale, mistyped, or is actually a product ID rather than a <code>ProductVariant</code> <code>gid</code>	Re-run Step 2 to get a fresh, real variant ID and paste it in exactly
The whole script fails with an authentication error	The cached Admin token from Lab 3.2 expired and the refresh failed, or <code>.env</code> isn't being loaded	Re-run <code>node test-admin.js</code> (Step 1) to confirm the token helper still works on its own
Entering the storefront password redirects to the plain homepage instead of the invoice	The browser wasn't already unlocked, so the password gate fired on the invoice link itself and dropped the destination	Unlock the store at its root URL first (Step 5), <i>then</i> open the same <code>invoiceUrl</code> again in that browser
The customer query fails with an approval-style error	Protected customer data access was never requested for this app	Complete that step from Lab 3.2 at <code>partners.shopify.com</code> before retrying
Fetching the draft order back (Step 4) returns <code>null</code>	The wrong ID was used — a product/variant ID instead of the <code>DraftOrder</code> <code>gid</code> from Step 3's response	Copy <code>data.draftOrderCreate.draftOrder.id</code> exactly, not any other ID from that response

Where This Goes Next

This is a genuinely different tool from Lab 3.3's Cart API: `checkoutUrl` is for a shopper's own live self-checkout, while `draftOrderCreate` and its `invoiceUrl` are for a merchant- or agent-initiated sale built on the customer's behalf — a phone order, or an admin-side agent assembling one from a conversation. Both patterns get wrapped as real Claude tools in Lab 3.6, and this exact draft-order mechanic is reused directly by Module 5's admin-scoped operations agent.